GENERATIVE AI & LLMs

A Beginner-Friendly Learning Guide

Concepts • How they work • Practical examples • Code • Learning roadmap

What this guide covers

Prompt Engineering, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), LangChain, LlamaIndex, Hugging Face, OpenAI APIs, and Vector Databases.

Designed for beginners who want to understand the concepts and gradually build practical GenAI applications.

1. How the topics fit together

These subjects are connected. An LLM generates language; prompt engineering guides it; a vector database can retrieve relevant information; RAG supplies that information to the LLM; frameworks such as LangChain and LlamaIndex help coordinate the steps; Hugging Face provides models and tools; and an API lets your application communicate with a model.

Your question / task
↓
Prompt Engineering Write clear instructions and provide context
↓
Optional RAG pipeline Retrieve useful passages from a knowledge base
↓
LLM Generate an answer using the prompt and supplied context
↓
Application response Show the answer, cite sources, or trigger an approved action

A simple example

Imagine building a college-policy assistant. The user asks, “What is the attendance requirement?” The app searches approved college documents, retrieves the relevant paragraph, gives that paragraph to the LLM with clear instructions, and returns an answer with the document name and page reference.

2. Prompt Engineering

Prompt engineering is the practice of writing and improving instructions so an AI model produces useful, relevant, and appropriately formatted responses. A prompt is the input you give the model.

Why it matters

- A vague prompt often produces a broad or inconsistent answer.
- A clear prompt defines the task, audience, context, limits, and expected output.
- Good prompts reduce rework, but they do not guarantee factual correctness.

A reliable prompt structure

- **Role:** Tell the model what perspective to use, such as “You are a beginner Python tutor.”
- **Task:** State exactly what it must do.
- **Context:** Supply the data, background, or situation it needs.
- **Constraints:** Set boundaries, such as “Use only the supplied data.”
- **Output format:** Ask for bullets, a table, JSON, or step-by-step explanation.
- **Quality check:** Ask it to identify assumptions or say when information is missing.

Weak prompt vs improved prompt

Weak: “Explain SQL.”

Improved: “Act as a beginner SQL tutor. Explain INNER JOIN using two small tables named customers and orders. Show the SQL query, expected result, and one common mistake. Use simple language.”

Common prompting techniques

- **Zero-shot:** Ask for a task without examples.
- **Few-shot:** Give a few examples of the input and desired output.
- **Step-by-step prompting:** Ask for a clear sequence of steps or a concise rationale appropriate to the task.
- **Structured output:** Specify a schema or fields when the application needs predictable output.
- **Iterative prompting:** Review the result, identify what is missing, and refine the prompt.

Practical example

```
Prompt:
You are a data analyst.
Explain the following KPI to a non-technical manager:
"Customer retention rate".
Include:
1. Meaning
2. Formula
3. A numerical example
4. One business use
Keep the answer under 150 words.
```

Remember

Prompt engineering improves how you communicate with a model. It is not the same as training a model, and a polished prompt cannot replace reliable source data or testing.

3. Large Language Models (LLMs)

A Large Language Model is an AI model trained on large amounts of text (and, in some systems, other types of data) to understand and generate language. It generates text by predicting likely next tokens based on the context it receives.

Important terms

- **Token:** A piece of text, such as a word, part of a word, or punctuation mark.
- **Training:** The process in which model parameters are adjusted using training data.
- **Inference:** Using a trained model to generate an output.
- **Context window:** The amount of input and conversation the model can consider at one time.
- **Temperature:** A generation setting that affects randomness; lower values are generally more predictable.
- **Hallucination:** An answer that sounds confident but is false, unsupported, or invented.

How an LLM works at a high level

- Text is split into tokens and converted into numerical representations.
- A transformer neural network processes relationships between tokens using attention mechanisms.
- The model estimates probabilities for the next token.
- A decoding method selects tokens repeatedly until the response is complete or a limit is reached.

What LLMs are good at

- Summarising and rewriting text.
- Explaining concepts and answering questions.
- Generating and reviewing code.
- Extracting or classifying information from text.
- Supporting conversational interfaces and content workflows.

Limitations and safe use

- The model may make up facts, citations, APIs, or code details.
- Its knowledge may be incomplete or out of date unless connected to current sources or tools.
- It can reflect bias in data and may misunderstand ambiguous instructions.
- Do not send confidential or personal data unless the system and organisation explicitly permit it.
- Validate important outputs with trusted sources, tests, and human review.

Key distinction

An LLM does not automatically know your private company files. You must provide relevant information in the prompt, connect an approved retrieval system, or use another authorised data integration.

4. Retrieval-Augmented Generation (RAG)

RAG is a design pattern that connects an LLM to an external knowledge source. Before answering, the application searches for relevant information and supplies the retrieved content to the model.

Why use RAG?

- Answer questions about private or organisation-specific documents.
- Use information that changes more frequently than a model's training data.
- Show references to the source passages used in an answer.
- Reduce unsupported answers by grounding the model in retrieved evidence. RAG does not eliminate hallucinations.

Typical RAG workflow

- **1. Collect:** Load PDFs, Word files, web pages, manuals, or database records that you are authorised to use.
- **2. Extract:** Convert the source material into text and retain useful metadata such as file name and page.
- **3. Chunk:** Split long content into smaller, overlapping sections.
- **4. Embed:** Convert each chunk into a numerical vector representing its semantic meaning.
- **5. Store:** Save vectors and metadata in a vector database or vector index.
- **6. Retrieve:** Convert the user's question into a vector and find relevant chunks.
- **7. Generate:** Add the retrieved chunks to the prompt and ask the LLM to answer from them.
- **8. Evaluate:** Check relevance, factual support, citations, latency, and cost.

Simple example

Question: “How many days do employees get for parental leave?”

The retriever searches the HR policy documents and returns the relevant policy paragraphs. The LLM is instructed to answer only from those paragraphs and to say when the policy does not contain the answer.

Important RAG design choices

- **Chunk size:** Very small chunks may lose context; very large chunks may contain distracting material.
- **Overlap:** A small overlap can preserve meaning across chunk boundaries.
- **Top-k:** The number of chunks returned for a query.
- **Metadata filtering:** Restrict retrieval by department, document type, date, or access permissions.
- **Hybrid search:** Combine semantic vector search with keyword search.
- **Reranking:** Reorder retrieved passages using a relevance model before generation.

RAG is not fine-tuning

RAG retrieves external information at question time. Fine-tuning changes model parameters through additional training. They solve different problems and can sometimes be used together.

5. Vector Databases

A vector database stores and searches numerical vectors, often called embeddings. Embeddings represent the meaning or characteristics of text, images, audio, or other data in a numerical space.

What is an embedding?

An embedding model converts an item into a list of numbers. Items with similar meaning are generally placed near one another in the embedding space. For example, “car” and “automobile” may have vectors that are closer than “car” and “banana”.

How similarity search works

- Create an embedding for each document chunk and store it with the original text and metadata.
- Create an embedding for the user's question.
- Compare the query vector with stored vectors using a similarity or distance measure.
- Return the most relevant matches, optionally applying metadata filters.

Common similarity measures

- **Cosine similarity:** Measures the angle between two vectors; often used for semantic similarity.
- **Dot product:** Multiplies corresponding vector values and sums them; interpretation depends on embedding normalisation.
- **Euclidean distance:** Measures straight-line distance between vectors.

Examples of vector database technologies

- **FAISS:** A library for efficient similarity search; often embedded within an application rather than operated as a full database service.
- **Chroma:** A developer-friendly option commonly used for local prototypes and applications.
- **Pinecone:** A managed vector database service.
- **Weaviate:** A vector database with search and data-management capabilities.
- **Milvus:** An open-source vector database designed for large-scale vector search.
- **pgvector:** Adds vector similarity search capabilities to PostgreSQL.

Beginner mental model

A normal database often answers “find rows where department = Sales”. A vector search system can help answer “find the policy paragraphs that mean something similar to this question”, even when the wording differs.

> **Important**
>
> A vector database does not understand meaning by itself. The embedding model creates the vectors; the database stores and searches them. Retrieval quality depends on both.

6. Hugging Face

Hugging Face is an AI platform and open-source ecosystem. It provides model repositories, datasets, libraries, demos, and collaboration tools that help developers find, use, share, and evaluate machine-learning models.

Main parts to know

- **Model Hub:** A catalogue of pretrained models for language, vision, audio, and other tasks.
- **Datasets:** A library and hub for discovering and loading datasets.
- **Transformers:** A popular Python library for using pretrained transformer models.
- **Tokenizers:** Tools that convert text into tokens and model inputs.
- **Spaces:** A way to host interactive machine-learning demos and applications.
- **Inference:** Ways to run models locally or through supported hosted services.

Simple Python example

```
# Install first:
# pip install transformers torch

from transformers import pipeline

classifier = pipeline("sentiment-analysis")
result = classifier("I enjoyed learning about language models.")
print(result)
```

The pipeline downloads or loads a suitable pretrained model and runs sentiment analysis. The first run may take time because model files may need to be downloaded. Some models require substantial memory or a GPU.

How to choose a model

- Check the model card for intended use, limitations, licence, and evaluation results.
- Check language support, model size, hardware requirements, and inference speed.
- Test it on examples similar to your actual task.
- Confirm that the licence and data-handling terms fit your project.

7. LangChain

LangChain is a framework for building applications powered by language models. It provides components and patterns for connecting models to prompts, retrievers, tools, memory-like state, and other services.

What it helps you build

- Prompt templates that accept changing input values.
- Model calls to supported providers.

- Document loading, text splitting, embeddings, and retrieval workflows.
- Chains or sequences of application steps.
- Tool-using assistants and agent-style workflows.

Conceptual example

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
"Explain {topic} to a beginner in three bullet points."
)

# A model is connected to the prompt in a runnable pipeline.
# Exact model setup depends on the provider and package version.
```

When should you use it?

Use LangChain when you need reusable building blocks to connect several parts of an LLM application. For a very small app, a direct model API call may be simpler. Framework APIs change, so follow the current official documentation and pin package versions for reproducible projects.

Do not confuse

LangChain is an application-development framework. It is not an LLM, not a vector database, and not an embedding model.

8. LlamaIndex

LlamaIndex is a framework focused on connecting LLMs with external data. It helps developers ingest, organise, index, retrieve, and query data such as PDFs, documents, databases, and other knowledge sources.

Main concepts

- **Data connectors:** Bring data into the application from supported sources.
- **Documents and nodes:** Represent source documents and smaller pieces of content.
- **Indexes:** Organise data to support retrieval and querying.
- **Retrievers:** Find relevant pieces of data for a question.
- **Query engines:** Combine retrieval and response synthesis.
- **Agents:** Allow an LLM-driven workflow to choose tools or actions where configured.

Conceptual example

```
# Illustrative workflow; exact imports can vary by version.
# 1. Load authorised documents.
# 2. Split documents into nodes.
# 3. Build an index using an embedding model.
# 4. Create a query engine.
# 5. Ask a question and inspect the retrieved sources.
```

When is it useful?

LlamaIndex is a strong option when the main challenge is making many kinds of data accessible to an LLM, especially for document question-answering and retrieval-heavy applications.

LangChain vs LlamaIndex

Both can support RAG and LLM applications. LangChain is commonly used for connecting application components and workflows; LlamaIndex puts particular emphasis on data ingestion, indexing, and retrieval. There is overlap, and you do not need to learn both at the same time.

9. OpenAI APIs

An API (Application Programming Interface) lets one software application communicate with another. An OpenAI API lets your code send requests to supported OpenAI models and receive outputs, subject to the API's current capabilities, account access, and usage terms.

How a typical API request works

- Create an API account and configure billing or project access as required.
- Create an API key and store it securely, such as in an environment variable.
- Install the official SDK or use an authorised HTTP client.
- Send the model and input instructions to the relevant endpoint.
- Read the response, handle errors, and display or process the result.

Python example

```
# Install the current official SDK:
# pip install openai

import os
from openai import OpenAI

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.responses.create(
model="YOUR_AVAILABLE_MODEL",
input="Explain an API to a beginner in two sentences."
)

print(response.output_text)
```

Replace the model placeholder with a model available to your API project. API names and features can change; use the current official API documentation when implementing a real application.

Security and reliability

- Never paste an API key into public code, GitHub, screenshots, or a frontend application.
- Use environment variables or a secrets manager; rotate a key if it is exposed.
- Set sensible usage limits and monitor cost.
- Handle rate limits, timeouts, invalid requests, and service errors.
- Avoid sending sensitive information unless you have permission and the correct safeguards.

API vs ChatGPT website

The ChatGPT product and the developer API are separate interfaces and may have separate billing, settings, and availability. Having access to one does not automatically mean the other is included.

10. Quick comparison: what each topic does

Topic	Main purpose	Think of it as
Prompt Engineering	Give the model clear instructions	How you ask
LLM	Understand and generate language	The language engine
RAG	Ground answers in retrieved external information	Look up, then answer
Vector Database	Store and search embeddings	Meaning-based search store
Hugging Face	Discover and run models, datasets, and tools	AI model ecosystem
LangChain	Connect LLM components and workflows	Application toolkit
LlamaIndex	Connect and retrieve external data for LLMs	Data and retrieval toolkit
OpenAI API	Call supported models from your code	A software connection

11. Beginner project: College Policy Question-Answering Assistant

Build one small project that uses the topics together. A user asks questions about a set of college documents and receives an answer grounded in the documents.

Project objective

Create a simple application that answers questions about attendance rules, exam eligibility, fee deadlines, and leave policies using only a small collection of approved sample documents.

Suggested architecture

- **Documents:** Use public sample policies or documents you have permission to process.
- **Ingestion:** Extract text and keep file name and page number as metadata.
- **Chunking:** Split the text into manageable sections.
- **Embeddings:** Use a suitable embedding model, for example from a supported provider or Hugging Face.
- **Vector storage:** Start locally with Chroma or FAISS; consider a hosted option later.
- **Retrieval:** Find the most relevant sections for the user's question.
- **Generation:** Call an LLM through an API and instruct it to answer from retrieved content only.
- **Interface:** Build a simple Streamlit or web interface.
- **Evaluation:** Prepare 20–30 questions with expected source passages; measure retrieval relevance and answer correctness.

Example system instruction

```
You answer questions about the supplied college policy excerpts.
Use only the supplied excerpts.
If the answer is not present, say:
"I could not find this information in the supplied documents."
Do not invent dates, fees, or rules.
Mention the source document and page when available.
```

What to include in your portfolio

- A clear README with setup instructions and architecture diagram.
- A small, legally usable sample document collection.

- A requirements file with tested package versions.
- Screenshots of the application and example questions.
- A test set showing correct answers, failed retrievals, and improvements.
- A section describing limitations, privacy, and future improvements.

12. Recommended learning roadmap

Stage 1 — Foundations

Learn what LLMs, tokens, embeddings, context windows, and hallucinations are.

Stage 2 — Prompting

Practise role, task, context, constraints, examples, and structured output.

Stage 3 — API basics

Make a simple Python API call; learn keys, environment variables, errors, and cost controls.

Stage 4 — Embeddings and vector search

Create embeddings, store them, and retrieve similar text.

Stage 5 — RAG

Build a document question-answering pipeline and return source references.

Stage 6 — Frameworks

Rebuild the same small RAG workflow with either LangChain or LlamaIndex.

Stage 7 — Hugging Face

Explore model cards, run a pretrained pipeline, and compare model behaviour.

Stage 8 — Evaluation

Test retrieval and answers, record failure cases, and improve the system.

A practical four-week schedule

- **Week 1:** LLM fundamentals and prompt engineering; practise 10 prompts for summarisation, extraction, and explanation.
- **Week 2:** API basics and Hugging Face; create a small text-classification or summarisation demo.
- **Week 3:** Embeddings, vector databases, and RAG; build document search.
- **Week 4:** Add a simple interface, evaluate with test questions, document the project, and publish a clean repository.

13. Beginner glossary

AI	Artificial Intelligence — systems that perform tasks associated with intelligent behaviour.
Machine Learning	A field of AI where systems learn patterns from data.
Deep Learning	Machine learning based on multi-layer neural networks.
Transformer	A neural-network architecture widely used in modern language models.
Token	A unit of text processed by a language model.
Embedding	A numerical vector representing information such as text meaning.
Chunk	A smaller section of a longer document.
Retriever	A component that finds relevant information for a query.
Inference	Running a trained model to produce a prediction or response.
Fine-tuning	Further training a pretrained model for a particular task or behaviour.
Agent	An LLM-based system configured to select tools or actions toward a goal.
Latency	The time taken for a system to respond.
Evaluation	Testing a model or application against defined quality criteria.

14. Practice questions

- What is the difference between an LLM and an API?
- Why can an LLM produce a hallucination?
- How does prompt engineering improve an answer?
- What are the main steps in a RAG pipeline?
- What is an embedding, and why is it useful?
- What is the difference between a vector database and a traditional relational database?
- When might you choose LangChain? When might LlamaIndex be a better fit?
- What information should you check on a Hugging Face model card?
- Why should API keys never be committed to GitHub?
- How would you test whether a document assistant gives answers supported by its sources?

Suggested self-check

You are ready to move to a small project when you can explain each topic in your own words, draw the RAG workflow, make one safe API call, and demonstrate retrieval from a small document collection.

15. Reliable learning resources

Use official documentation because libraries, API names, model availability, and recommended practices change.

- OpenAI platform documentation: https://platform.openai.com/docs
- Hugging Face documentation: https://huggingface.co/docs
- LangChain documentation: https://python.langchain.com/docs/
- LlamaIndex documentation: https://docs.llamaindex.ai/

- Chroma documentation: https://docs.trychroma.com/
- FAISS documentation: https://github.com/facebookresearch/faiss
- Pinecone documentation: https://docs.pinecone.io/

Learning tip: Do not try to memorise every framework. Understand the underlying ideas first, then build one working application and learn the framework features you actually need.